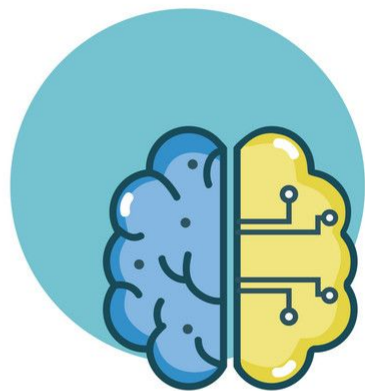


INTRODUCTION TO MACHINE LEARNING

UNSUPERVISED LEARNING



Elisa Ricci



UNSUPERVISED LEARNING

- We observe data from the (unknown) distribution $p_{\text{data}} \in \Delta(\mathcal{X})$
- We lack observations of a target variable (e.g. class labels)
- Implicit supervision is given via the design of the objective function

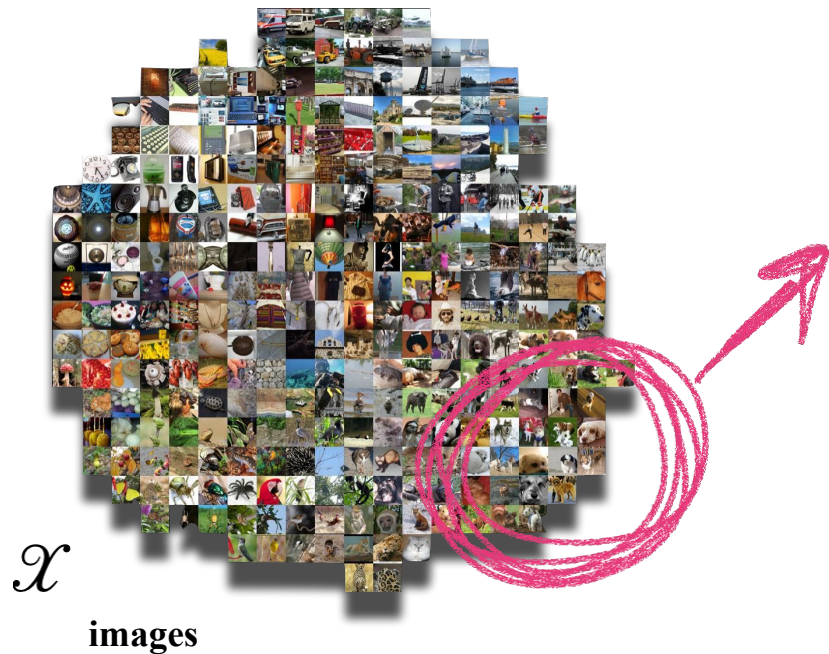
DIMENSIONALITY
REDUCTION

CLUSTERING

DENSITY
ESTIMATION

DIMENSIONALITY REDUCTION

Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ mapping each (high dimensional) input $x \in \mathcal{X}$ to a lower dimensional embedding $f(x) \in \mathcal{Y}$, where $\dim(\mathcal{Y}) \ll \dim(\mathcal{X})$.



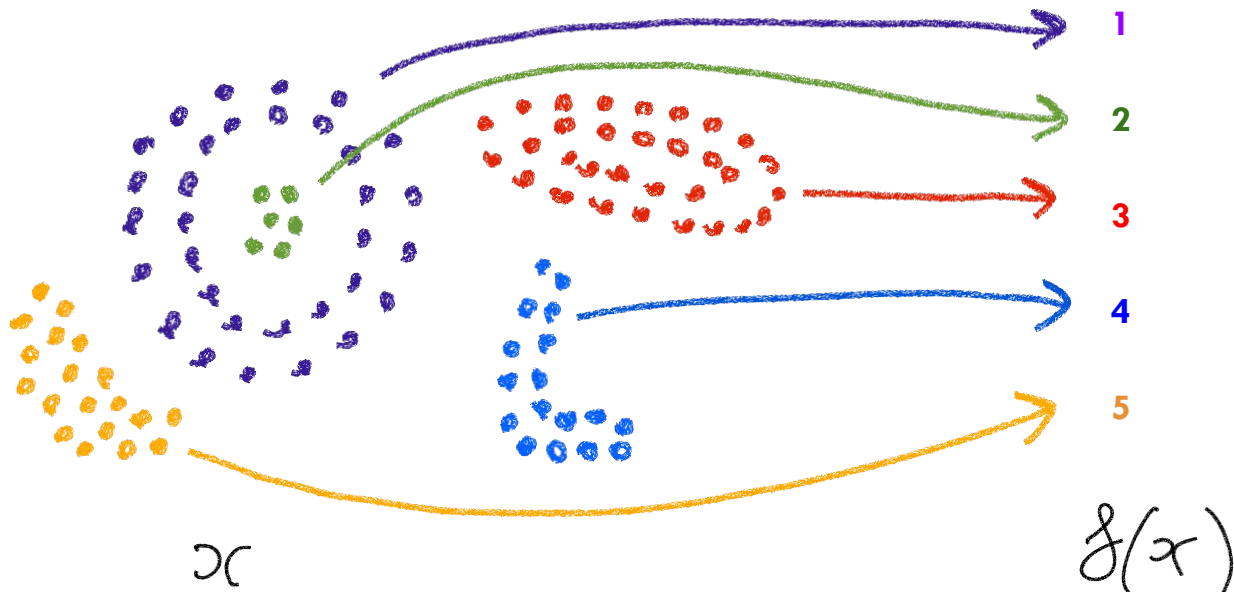
$$\mathcal{Y} = \mathbb{R}^2$$

DIMENSIONALITY REDUCTION: WHY?

- Compress the input data by **reducing the feature dimensionality**, while preserving as much information as possible. The way information loss is measured yields different algorithms
- This **reduces time** of subsequent data elaboration and/or **storage**
- Enables **better visualization** of data
- Reduces the **curse of dimensionality**

CLUSTERING (PARTITIONAL)

Find a function $f \in \mathbb{N}^{\mathcal{X}}$ that assigns each input $x \in \mathcal{X}$ a cluster index $f(x) \in \mathbb{N}$.
All points mapped to the same index form a cluster.

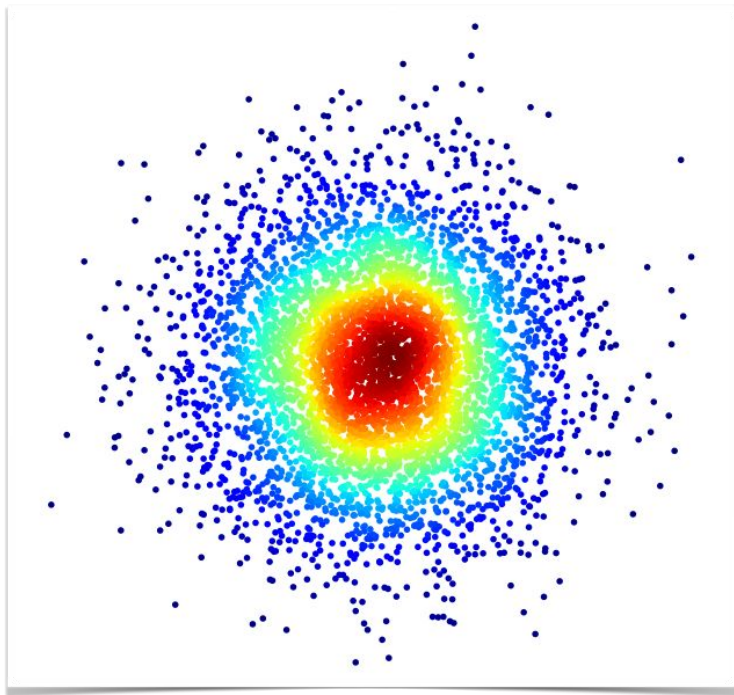


CLUSTERING: WHY?

- Allows to **analyze data by grouping together** data points that exhibit some regular pattern or similarity under some predefined criterion.
- Can be used to **compress data** by reducing the number of data points as opposed to reducing the feature dimensionality
- **Applications examples:** cluster users by preference (e.g based on movie ratings), group genes families from gene sequences, find communities in social networks, etc.

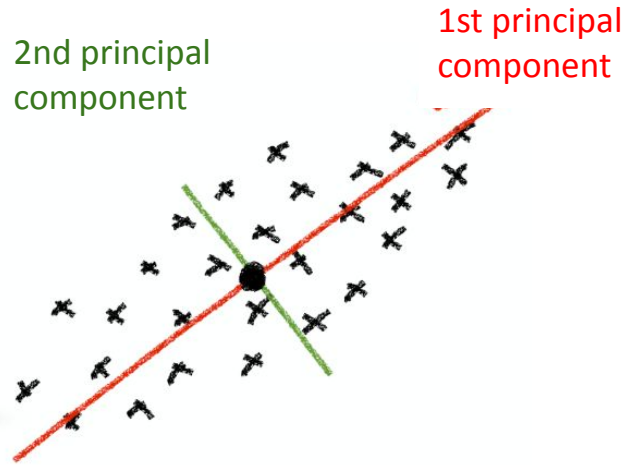
DENSITY ESTIMATION

Find a probability distribution $f \in \Delta(\mathcal{X})$ that fits the data $x \in \mathcal{X}$



DENSITY ESTIMATION: WHY?

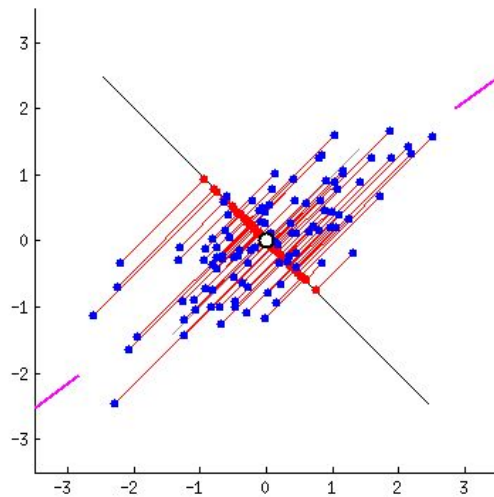
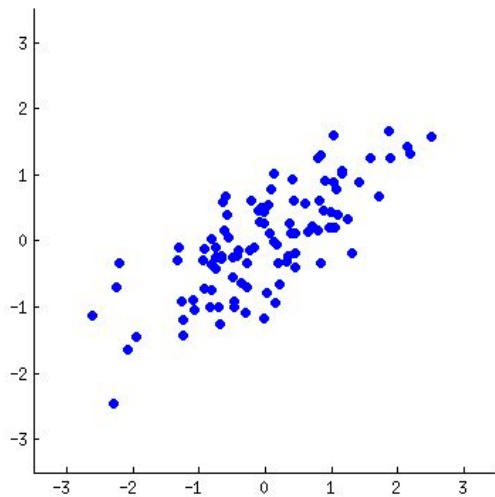
- Density estimation allows to get an **explicit estimate** of the unknown probability distribution that generated the training data.
- It enables the **generation of new data** by sampling from the estimated distribution
- It enables **detecting anomalies/novelities** in terms of data points that exhibit low probabilities according to the estimated distribution



PRINCIPAL COMPONENT ANALYSIS

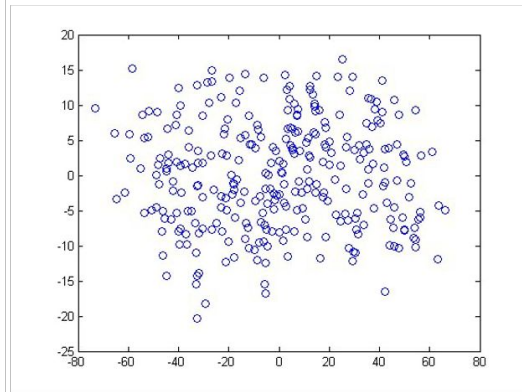
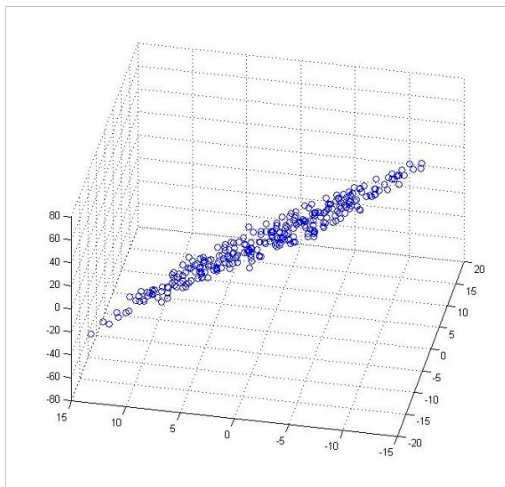
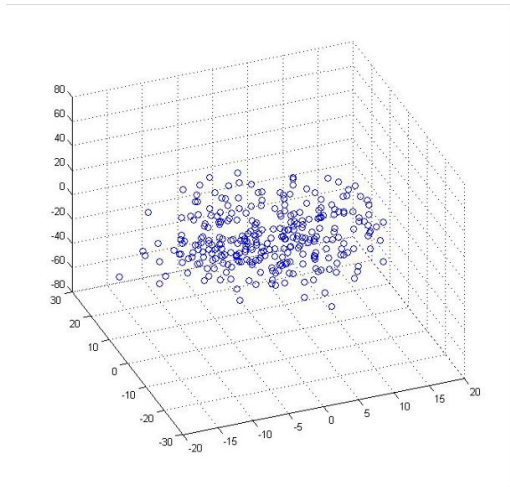
MAIN IDEA

- Find orthogonal directions of maximum variance
- Change coordinate system
- Drop dimensions of least variance



MAIN IDEA

Same from reducing from 3D to 2D



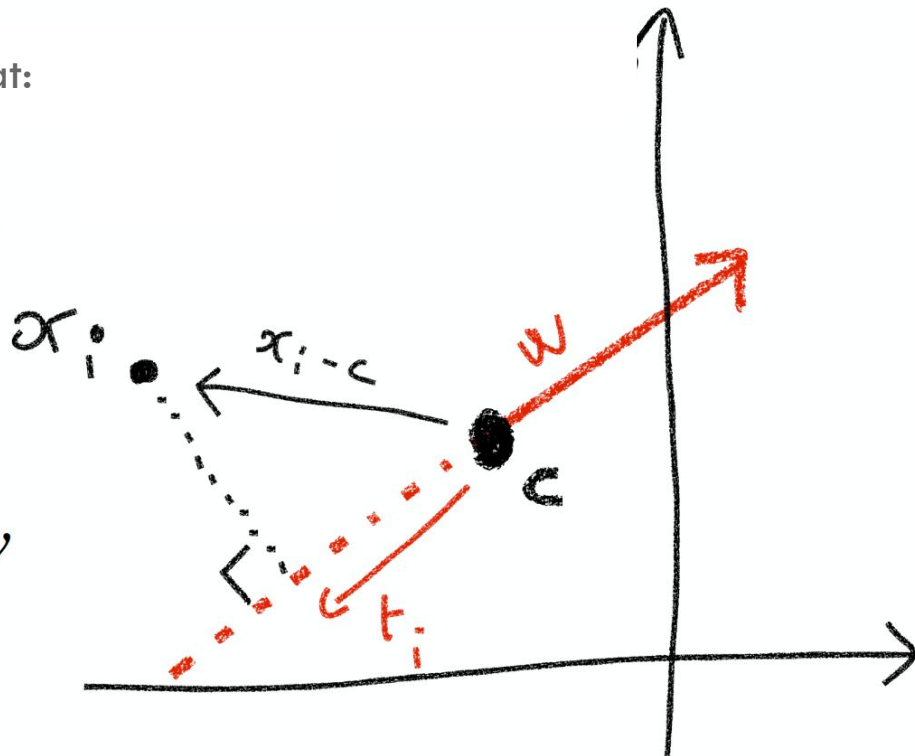
VARIANCE ALONG UNIT DIRECTION w

We consider w such that:

$$w^T w = 1$$

and compute:

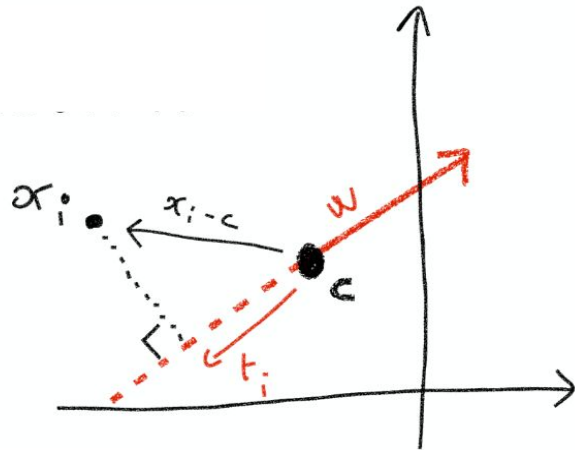
$$t_i = (x_i - c)^T w$$



VARIANCE ALONG UNIT DIRECTION $\mathbf{w}$

$$\mathbf{E}[t] = \frac{1}{n} \sum_{i=1}^n t_i = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \mathbf{c})^\top \mathbf{w}$$

$$\underbrace{\frac{1}{n} \sum_i \mathbf{x}_i}_{\mathbf{\bar{x}}} = \mathbf{\bar{x}}^\top \mathbf{w} - \mathbf{c}^\top \mathbf{w}$$



$$\mathbf{Var}[t] = \frac{1}{n} \sum_{i=1}^n (t_i - \mathbf{E}[t])^2 = \frac{1}{n} \sum_{i=1}^n [(\mathbf{x}_i - \mathbf{\bar{x}})^\top \mathbf{w}]^2$$

$$= \frac{1}{n} \sum_{i=1}^n (\mathbf{\bar{x}}_i^\top \mathbf{w})^2 = \mathbf{w}^\top \left[\frac{1}{n} \mathbf{\bar{X}} \mathbf{\bar{X}}^\top \right] \mathbf{w} = \mathbf{w}^\top \mathbf{C} \mathbf{w}$$

$$t_i = (\mathbf{x}_i - \mathbf{c})^\top \mathbf{w}$$

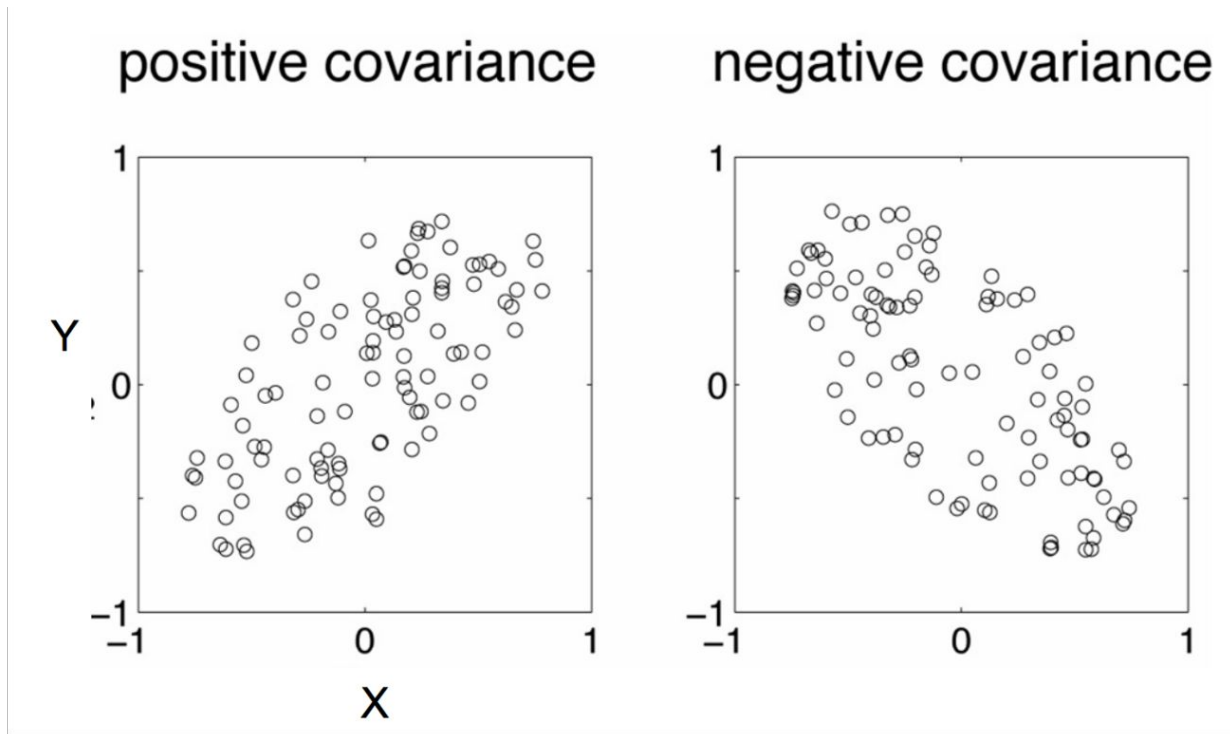
$$\mathbf{\bar{X}} = [\mathbf{\bar{x}}_1, \dots, \mathbf{\bar{x}}_n]$$

Covariance matrix

RECAP: COVARIANCE

- **Variance and Covariance:** Measure of the “**spread**” of a set of points around their center of mass (mean)
- **Variance:** Measure of the deviation from the mean for points in one dimension
- **Covariance:**
 - Measure of how much each of the dimensions vary from the mean with respect to each other
 - Covariance is measured between two dimensions
 - Covariance sees if there is a relation between two dimensions

RECAP: COVARIANCE



Positive: Both dimensions increase or decrease together

Negative: While one increase the other decrease

RECAP: EIGENVALUES AND EIGENVECTORS

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

$\mathbf{A}$: Square Matrix

$\mathbf{x}$: Eigenvector or characteristic vector

λ : Eigenvalue or characteristic value

RECAP: EIGENVALUES AND EIGENVECTORS

$$Ax = \lambda x$$

A: Square Matrix

x: Eigenvector or characteristic vector

λ : Eigenvalue or characteristic value

Example

Show $x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ is an eigenvector for $A = \begin{bmatrix} 2 & -4 \\ 3 & -6 \end{bmatrix}$

$$\text{Solution : } Ax = \begin{bmatrix} 2 & -4 \\ 3 & -6 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\text{But for } \lambda = 0, \lambda x = 0 \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Thus, x is an eigenvector of A , and $\lambda = 0$ is an eigenvalue.

RECAP: EIGENVALUES AND EIGENVECTORS

We can find eigenvalues solving

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0$$

Example

Find eigenvalues of $\mathbf{A}$:

$$\mathbf{A} = \begin{bmatrix} 2 & -12 \\ 1 & -5 \end{bmatrix}$$

$$\begin{aligned} |\lambda \mathbf{I} - \mathbf{A}| &= \begin{vmatrix} \lambda - 2 & 12 \\ -1 & \lambda + 5 \end{vmatrix} = (\lambda - 2)(\lambda + 5) + 12 \\ &= \lambda^2 + 3\lambda + 2 = (\lambda + 1)(\lambda + 2) \end{aligned}$$

Two eigenvalues: -1, -2

RECAP: EIGENVALUE DECOMPOSITION

- Given a symmetric matrix $A \in \mathbb{R}^{m \times m}$
- There exists $U = [\mathbf{u}_1, \dots, \mathbf{u}_m] \in \mathbb{R}^{m \times m}$ and $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_m)^\top \in \mathbb{R}^m$

such that

$$A = U\Lambda U^\top = \sum_{j=1}^m \lambda_j \mathbf{u}_j \mathbf{u}_j^\top \quad \text{and} \quad U^\top U = U U^\top = I$$

$$\Lambda = \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_m \end{bmatrix}$$

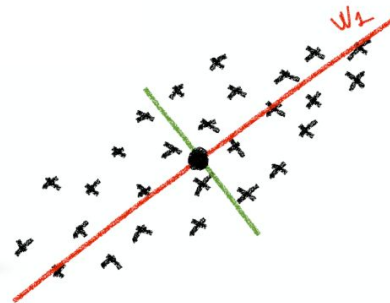
where I is the identity matrix, $\mathbf{u}_j$ is an eigenvector and λ_j is the corresponding eigenvalue

- Note: we assume descending ordering $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m$

FIRST PRINCIPAL COMPONENT

$$\mathbf{w}_1 \in \arg \max \{ \mathbf{w}^\top \mathbf{C} \mathbf{w} : \mathbf{w}^\top \mathbf{w} = 1 \}$$

Variance along $\mathbf{w}$

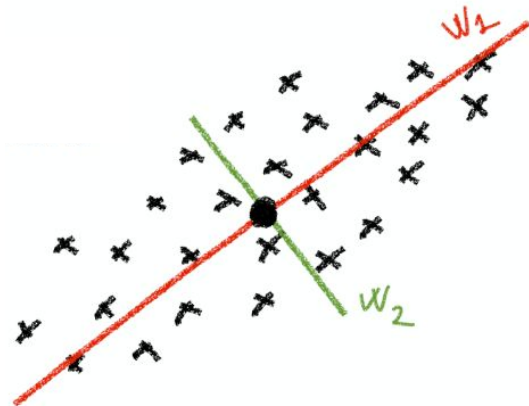


- It can be shown that the largest eigenvalue of $\mathbf{C}$ is the variance along the first principal component
- The first principal component $\mathbf{w}_1$ is the corresponding eigenvector

SECOND PRINCIPAL COMPONENT

$$\mathbf{w}_2 \in \arg \max \{ \mathbf{w}^\top \mathbf{C} \mathbf{w} : \mathbf{w}^\top \mathbf{w} = 1, \mathbf{w} \perp \mathbf{w}_1 \}$$

- It can be shown that the second largest eigenvalue of $\mathbf{C}$ is the variance along the second principal component
- The second principal component $\mathbf{w}_2$ is the corresponding eigenvector



i -TH PRINCIPAL COMPONENT

- The i -th largest eigenvalue of $\mathbf{C}$ is the variance along the i -th principal component
- The i -th principal component is the corresponding eigenvector

$$\mathbf{w}_i \in \arg \max \left\{ \mathbf{w}^\top \mathbf{C} \mathbf{w} : \mathbf{w}^\top \mathbf{w} = 1, \mathbf{w} \perp \mathbf{w}_j \text{ for } 1 \leq j < i \right\}$$

PCA USING EIGENVALUE DECOMPOSITION

Input: Data points $X = [\mathbf{x}_1, \dots, \mathbf{x}_n]$

Centering: $\bar{X} = X - \frac{1}{n} X \mathbf{1}_n \mathbf{1}_n^\top$

Handwritten notes:
- $\bar{x}$ mean data point (red arrow pointing to $\bar{X}$)
- $\mathbf{1}_n$ vector of ones (blue arrow pointing to $\mathbf{1}_n$)

Compute covariance matrix: $C = \frac{1}{n} \bar{X} \bar{X}^\top$

Eigenvalue decomposition:

$$U, \lambda = \text{eig}(C)$$

Output: Principal components $W = U = [\mathbf{u}_1, \dots, \mathbf{u}_m]$ and variances $\lambda = (\lambda_1, \dots, \lambda_m)$

PCA USING SINGULAR VALUE DECOMPOSITION (SVD)

SVD: Let $A \in \mathbb{R}^{m \times n}$. There exist $U \in \mathbb{R}^{m \times k}$, $s \in \mathbb{R}^k$, with $s_1 \geq \dots \geq s_k > 0$ and $V \in \mathbb{R}^{n \times k}$ such that:

$$A = USV^T \quad \text{and} \quad U^T U = V^T V = I$$

$$S = \begin{bmatrix} s_1 & & \\ & \ddots & \\ & & s_k \end{bmatrix}$$

PCA USING SINGULAR VALUE DECOMPOSITION (SVD)

Input: Data points $X = [\mathbf{x}_1, \dots, \mathbf{x}_n]$

Centering:

$$\bar{X} = X - \frac{1}{n} X \mathbf{1}_n \mathbf{1}_n^T$$

Handwritten notes:
- $\bar{x}$ mean data point (with arrow pointing to $\bar{X}$)
- $\mathbf{1}_n$ vector of ones (with arrow pointing to $\mathbf{1}_n$)

SVD decomposition:

$$U, s, V = \text{SVD}(\bar{X})$$

Output: Principal components $U = [\mathbf{u}_1, \dots, \mathbf{u}_k]$ and variances

$$\left(\frac{s_1^2}{n}, \dots, \frac{s_k^2}{n} \right)$$

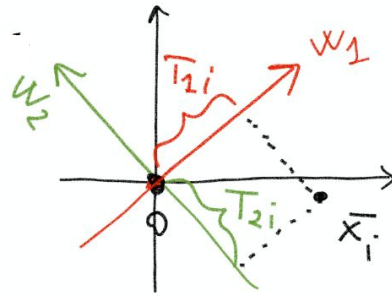
Handwritten notes:
- Because $\bar{X} = USV^T$ and
- $C = \frac{1}{n} \bar{X} \bar{X}^T = \frac{1}{n} USV^T VSU^T = U \boxed{\frac{S^2}{n}} U^T$ (with a green double-check mark)

DIMENSIONALITY REDUCTION USING PCA

- Let $\hat{W} = [\mathbf{w}_1, \dots, \mathbf{w}_k]$ hold the first k principal components derived from data points $\bar{X} = [\bar{\mathbf{x}}_1, \dots, \bar{\mathbf{x}}_n]$
- Change to reduced coordinate system with the k principal components as axis

Principal component scores

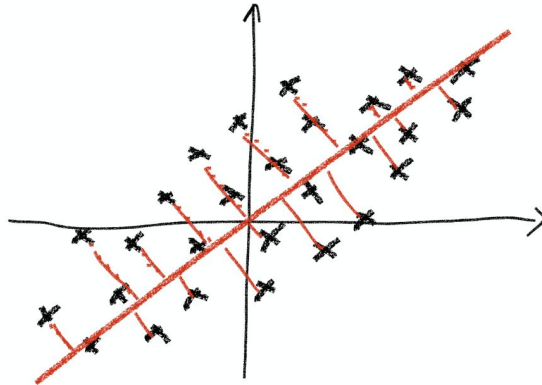
$$T = \hat{W}^T \bar{X} \in \mathbb{R}^{k \times n}$$



- We can use eigenvalue decomposition or SVD algorithms that avoid computing the full decompositions

ALTERNATIVE INTERPRETATION

- The first principal component can be interpreted as the line in space with **minimal squared distance** from the data points.
- A similar interpretation can be given to the other principal components



SCALING OF VARIABLES

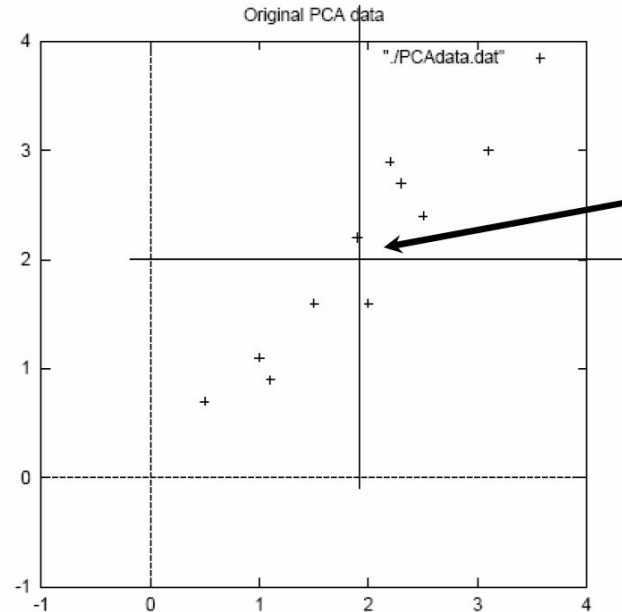
- The scale of the feature dimensions matters
- If features are expressed in different units it is recommended to scale them to have unit standard deviation

EXAMPLE

Step 1: Data Centering

DATA:

x	y
2.5	2.4
0.5	0.7
2.2	2.9
1.9	2.2
3.1	3.0
2.3	2.7
2	1.6
1	1.1
1.5	1.6
1.1	0.9



EXAMPLE

Step 2: Compute the covariance matrix $\mathbf{C}$

$$\mathbf{C} = \begin{bmatrix} 0.616 & 0.615 \\ 0.615 & 0.716 \end{bmatrix}$$

Since the non-diagonal elements in $\mathbf{C}$ are positive, both the x and y variable increase together.

EXAMPLE

Step 3: Calculate the eigenvectors and eigenvalues of the covariance matrix

$$U = \begin{bmatrix} -0.735 & -0.677 \\ 0.677 & -0.735 \end{bmatrix}$$

$$[\lambda_1 \ \lambda_2] = [0.04 \ 1.28]$$

EXAMPLE

Eigenvectors are plotted as diagonal dotted lines (they are perpendicular to each other)

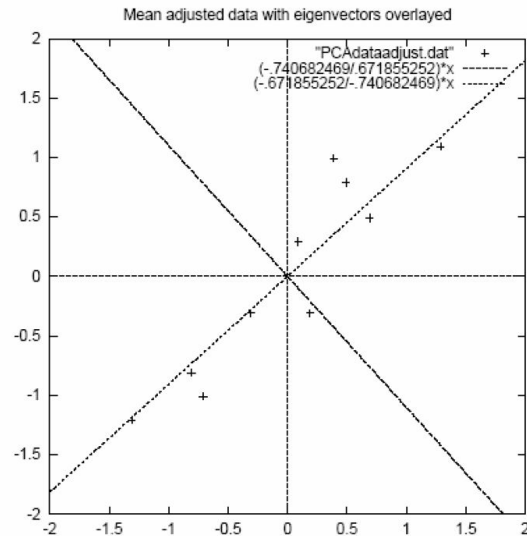


Figure 3.2: A plot of the normalised data (mean subtracted) with the eigenvectors of the covariance matrix overlayed on top.

EXAMPLE

Step 4:

We can either form a feature **vector** with both of the eigenvectors or we can choose to leave out the smaller, less significant component and only have a single column in U

We can derive new data coordinates multiplying the matrix with the eigenvectors in the columns transposed and the matrix of the centered data

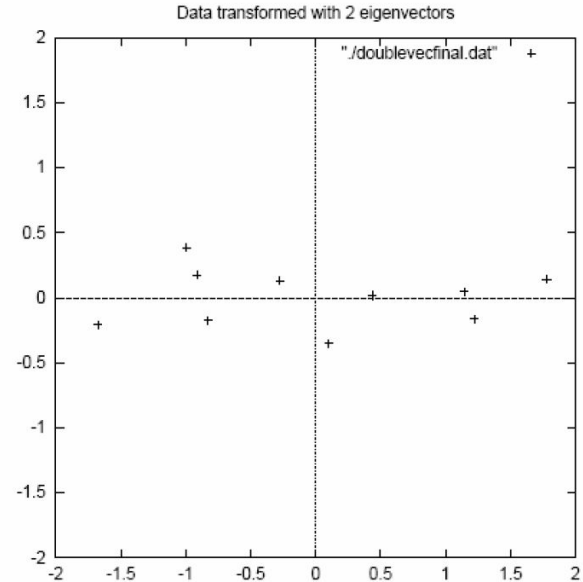
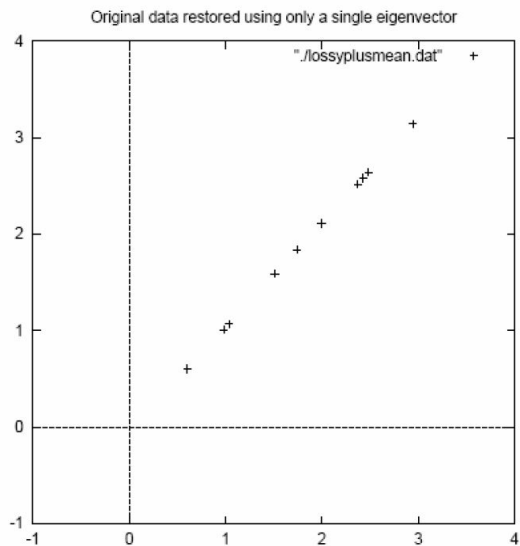
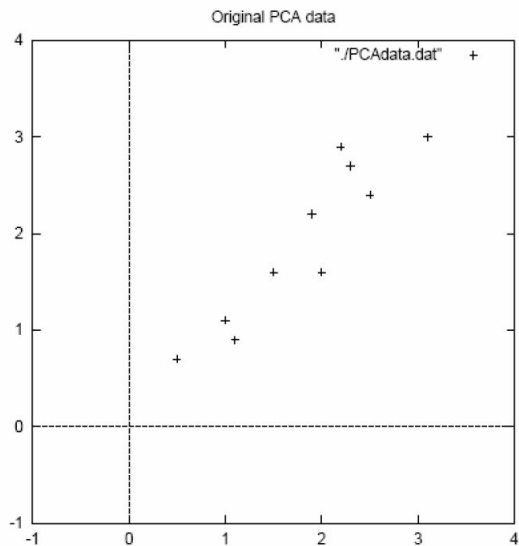


Figure 3.3: The table of data by applying the PCA analysis using both eigenvectors, and a plot of the new data points.

EXAMPLE

Reducing the dimensions...



HOW MANY PRINCIPAL COMPONENTS?

- The **number of components** for dimensionality reduction depends on the goal and application
- No means of validating it, unless we use it in the context of a supervised method (e.g. reduce the input dimensionality for a supervised algorithm)
- Nonetheless we can compute the **cumulative proportion of explained variance**, which for the first principal components is given by

EIGENVALUE
DECOMPOSITION

$$\frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^m C_{jj}}$$

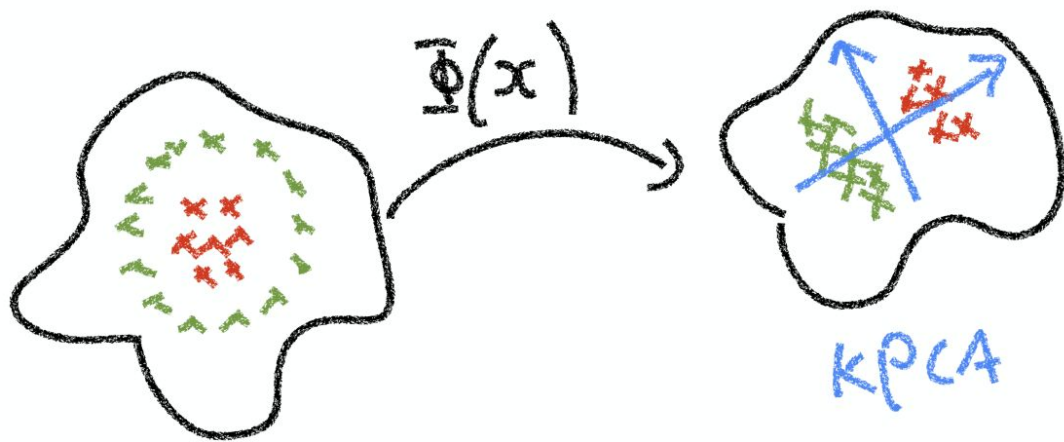
SVD

$$\frac{\sum_{j=1}^k s_j^2}{\sum_{ij} \bar{X}_{ji}^2}$$

- This allows to estimate the amount of information loss

KERNEL PCA (KPCA)

- PCA reduces the dimensionality via a linear transformation
- By using the kernel trick one can apply PCA in a higher dimensional space, yielding a non-linear transformation in the original space



OTHER DIMENSIONALITY REDUCTION TECHNIQUES

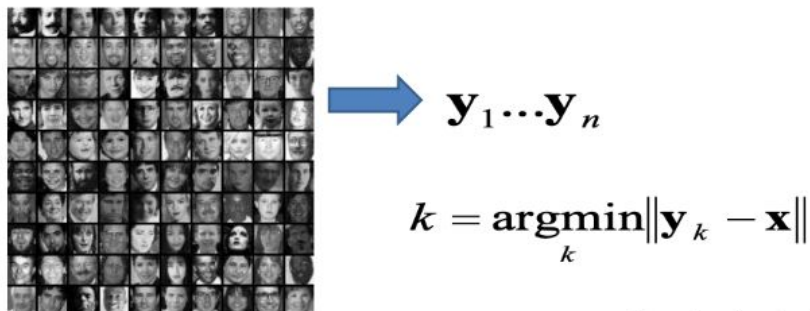
- **PCA** (Principal Component Analysis): Find projection that maximize the variance
- **Multidimensional Scaling**: Find projection that best preserves inter-point distances
- **LDA** (Linear Discriminant Analysis): Maximizing the component axes for class-separation

APPLICATION: FACE RECOGNITION

1. Treat face image as a vector of intensities



2. Recognize face by nearest neighbor in database



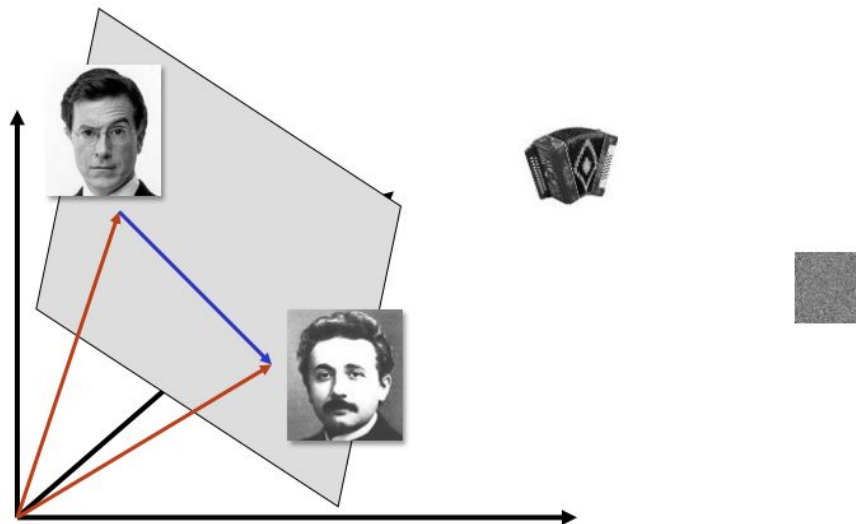
FACE IMAGES

- When viewed as vectors of pixel values, face images are extremely high-dimensional (e.g. 100×100 image = 10K dimensions)
- But very few 10,000-dimensional vectors are valid face images
- We want to effectively model the subspace of face images



THE SET OF FACES

- The set of faces is a “subspace” of the set of images
- We can find the best subspace using PCA

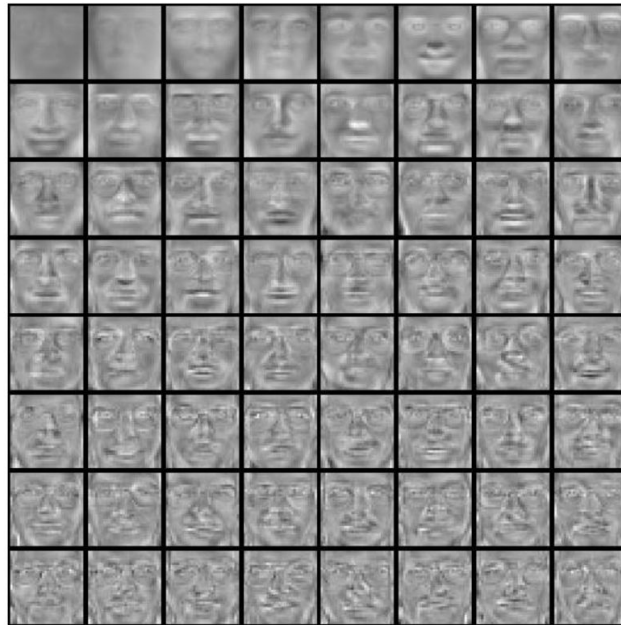


EIGENFACES

Mean: μ



Top eigenvectors: $u_1, \dots, u_k$



slide by Derek Hoiem

FACE REPRESENTATION AND RECONSTRUCTION

Face in the face space coordinates



$$\mathbf{x} \rightarrow [\mathbf{u}_1^T (\mathbf{x} - \mu), \dots, \mathbf{u}_k^T (\mathbf{x} - \mu)]$$

$$= t_1 \dots t_k$$

Reconstruction



=



+



$\hat{\mathbf{x}}$

=

μ

+

$\mathbf{u}_1 t_1$

...

μ

+

$\mathbf{u}_1 t_1$

+

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

RECONSTRUCTION

Different numbers of eigenvalues (ORL dataset)

$P = 4$



$P = 200$

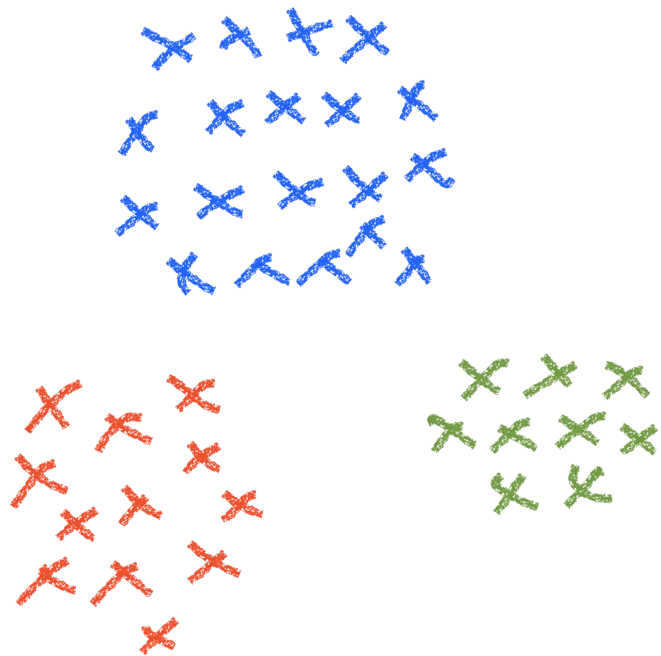


$P = 400$



ALGORITHM: DETECTION & RECOGNITION WITH EIGENFACES

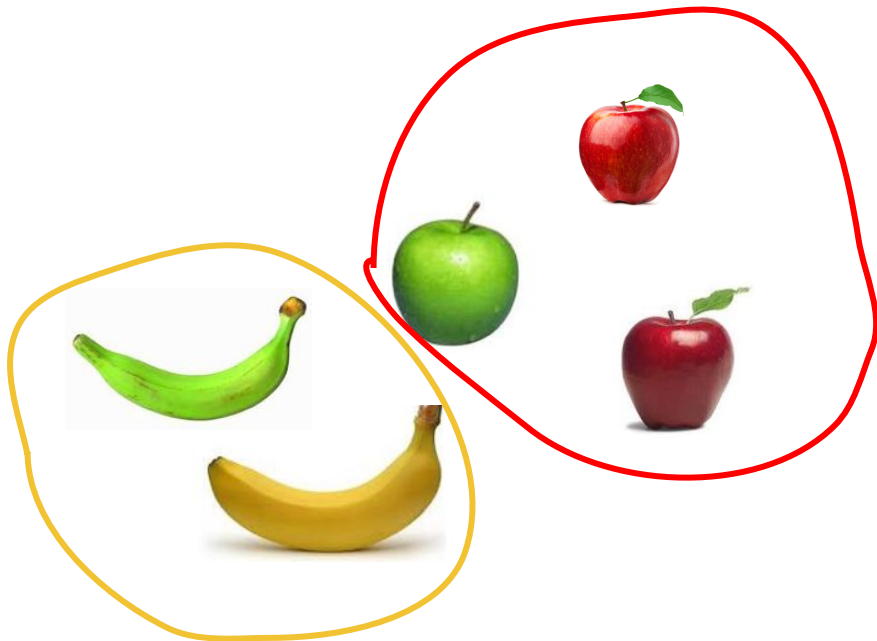
1. Process the image database (set of images with labels)
 - Run PCA: compute eigenfaces
 - Calculate K coefficients for each image
2. Given a new image (to be recognized) $\mathbf{x}$, calculate K coefficients
3. Detect if $\mathbf{x}$ is a face (distance between $\mathbf{x}$ and reconstructed image $<$ threshold)
4. If it is a face, who is it?
 - Find closest labeled face in database (KNN in K -D space)



K-MEANS CLUSTERING

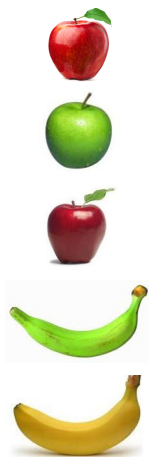
UNSUPERVISED LEARNING: CLUSTERING

Unsupervised learning setting: we are given data, i.e. examples, but no labels



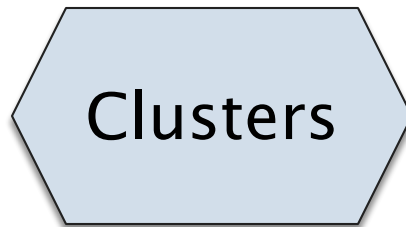
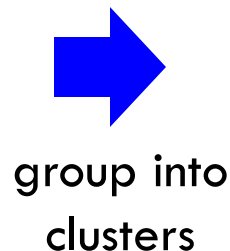
UNSUPERVISED LEARNING: CLUSTERING

Raw data



features

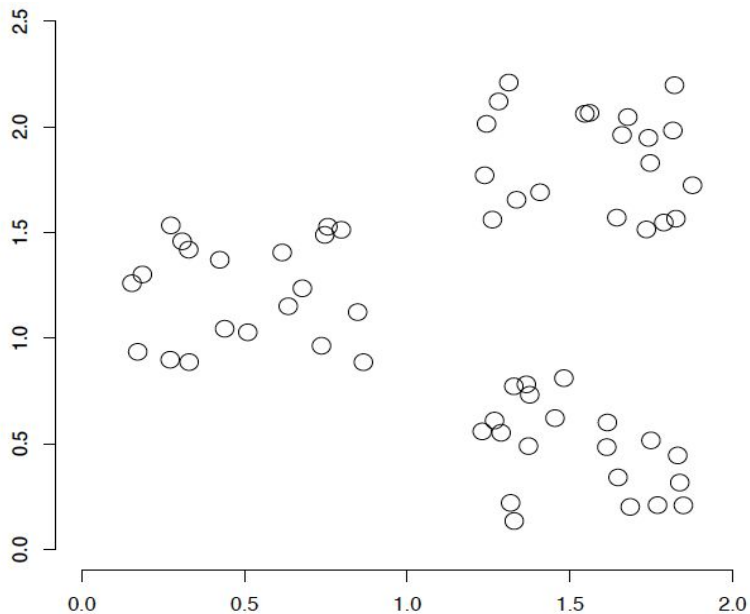
$f_1, f_2, f_3, \dots, f_n$
 $f_1, f_2, f_3, \dots, f_n$
 $f_1, f_2, f_3, \dots, f_n$
 $f_1, f_2, f_3, \dots, f_n$
 $f_1, f_2, f_3, \dots, f_n$



No supervision: we are only given data and want to find groupings

UNSUPERVISED LEARNING: CLUSTERING

Clustering: the process of grouping a set of objects into classes of similar objects

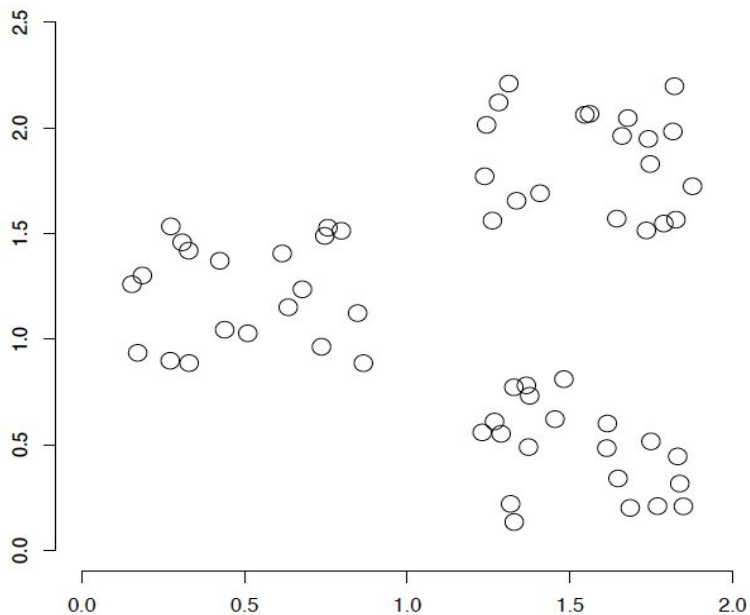


Which clustering algorithms can we use?

What are some of the issues for clustering?

UNSUPERVISED LEARNING: CLUSTERING

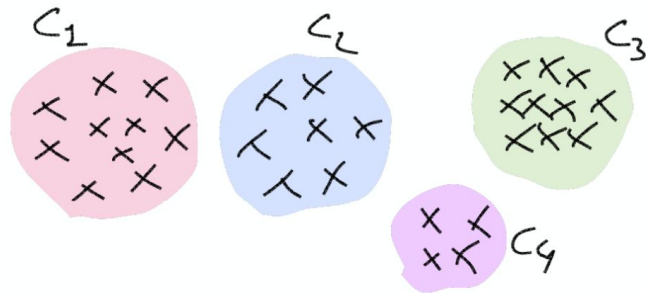
Clustering: the process of grouping a set of objects into classes of similar objects



Which clustering algorithms can we use?

Most well-known and popular clustering algorithm is **K-Means**

K-MEANS CLUSTERING



- Data points $X = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{d \times n}$
- Fix a number of clusters k
- Find a partition of data points into k sets minimizing the variation within each set

$$\min_{\mathcal{C}_1, \dots, \mathcal{C}_k} \sum_{j=1}^k V(\mathcal{C}_j)$$

- The variation is typically given by $V(\mathcal{C}_j) = \sum_{i \in \mathcal{C}_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$
- The centroid is computed with

$$\boldsymbol{\mu}_j = \frac{1}{|\mathcal{C}_j|} \sum_{i \in \mathcal{C}_j} \mathbf{x}_i$$

ALGORITHM

Initialization: Use some initialization strategy to get some initial cluster centroids $\mu_1, \dots, \mu_k$

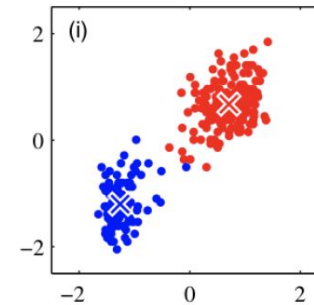
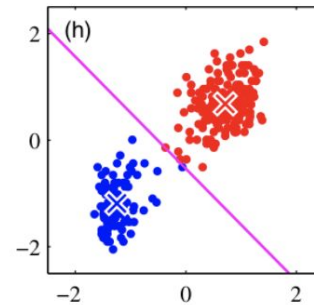
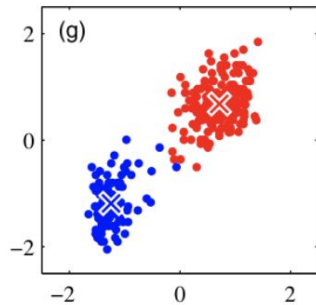
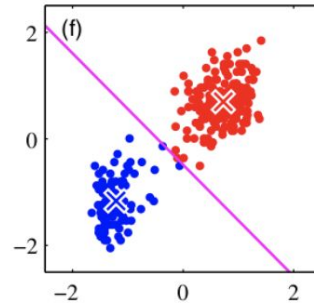
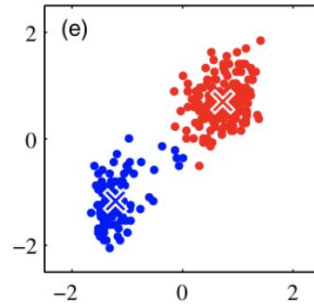
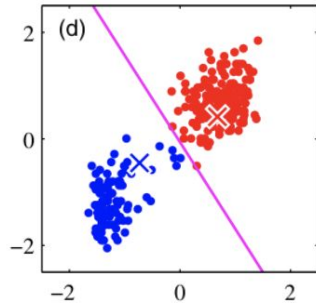
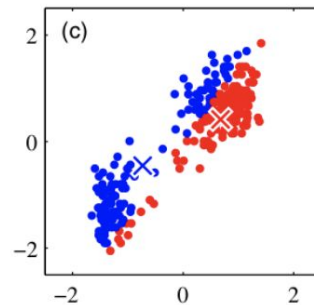
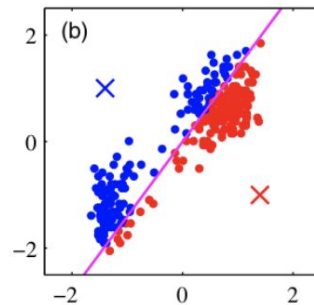
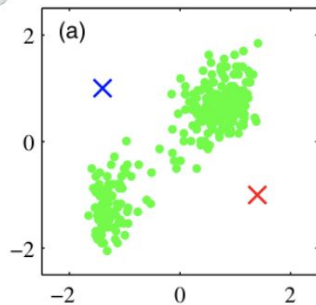
while clusters change **do**

Assign each datapoint to the closest centroid forming new clusters, i.e.

$$\mathcal{C}_j = \{i \in \{1, \dots, n\} : j = \arg \min_{\ell} \|x_i - \mu_{\ell}\|\}$$

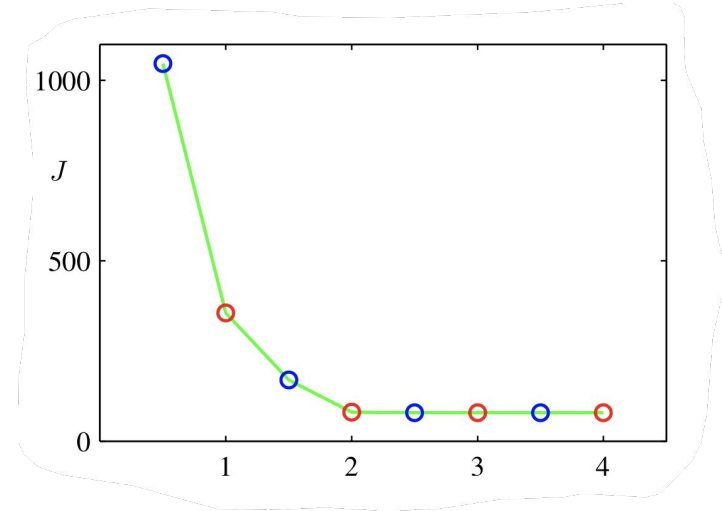
Compute cluster centroids $\mu_1, \dots, \mu_k$

EXAMPLE



PROPERTIES

- **Sensitive to the scale of features.**
Feature normalization is required in case of features with different scales
- Algorithm is **guaranteed to converge**, because it strictly improves the objective if there is at least a cluster change and the set of possible partitions is finite
- It is not guaranteed to find the global minimum but a **local one**.



Value of objective function for previous example

PROPERTIES OF K-MEANS

Algorithm is guaranteed to converge, because it strictly improves the objective if there is at least a cluster change and the set of possible partitions is finite

Does each step of k-means move towards reducing the loss function (or at least not increasing it)?

Idea:

- Step 1 (**assignment**): any other assignment would end up in a larger loss
- Step 2 (**centroid computation**): the mean of a set of values minimizes the squared error

PROPERTIES OF K-MEANS

It is not guaranteed to find the global minimum but a local one.

Does this mean that k-means will always find the minimum loss/clustering?

- NO! It will find *a minimum*.
- Unfortunately, the k-means loss function is generally not convex and for most problems has many minima
- We are only guaranteed to find one of them

INITIAL CENTROID SELECTION

- Results can vary drastically based on random seed selection
- Some seeds can result in poor convergence rate or convergence to suboptimal clusterings
- Common heuristics
 - Random points (not examples) in the space
 - Randomly pick examples
 - Points least similar to any existing center (furthest centers heuristic)
 - Try out multiple starting points
 - Initialize with the results of another clustering method

RUNNING TIME

Alternate:

- Step 1 (**assignment**): any other assignment would end up in a larger loss

$O(kn)$ time

- Step 2 (**centroid computation**): the mean of a set of values minimizes the squared error

$O(n)$ time

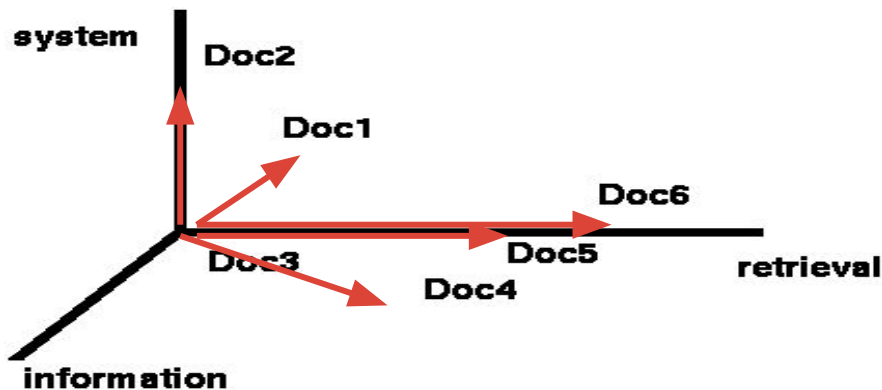
ISSUES WITH K-MEANS: DISTANCE

Euclidean distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

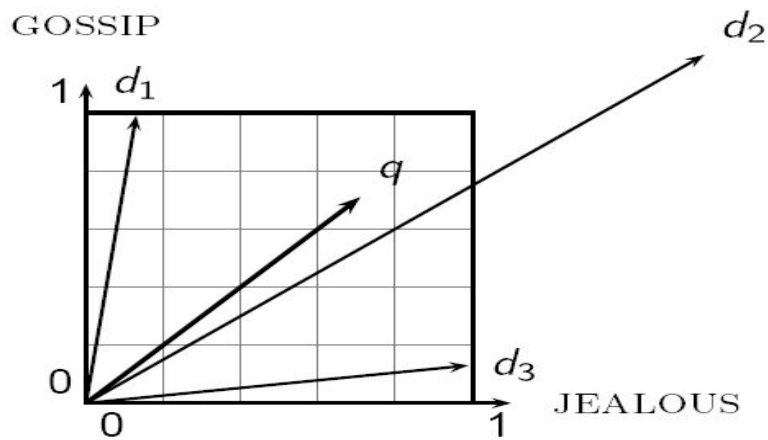
ISSUES WITH K-MEANS: DISTANCE

- Is Euclidean Distance always a good choice?
- Example: Clustering documents
 - One feature for each word. The value is the number of times that word occurs.
 - Documents are points or vectors in this space



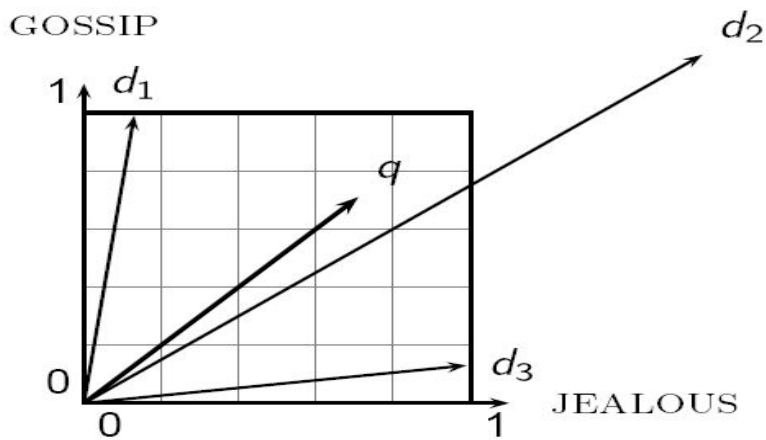
ISSUES WITH K-MEANS: DISTANCE

- Euclidean distance does not work here
- Which document is closest to q using Euclidean distance?



ISSUES WITH K-MEANS: DISTANCE

- Euclidean distance does not work here
- The Euclidean distance between q and d_2 is large but the distributions of terms in q and d_2 are very similar

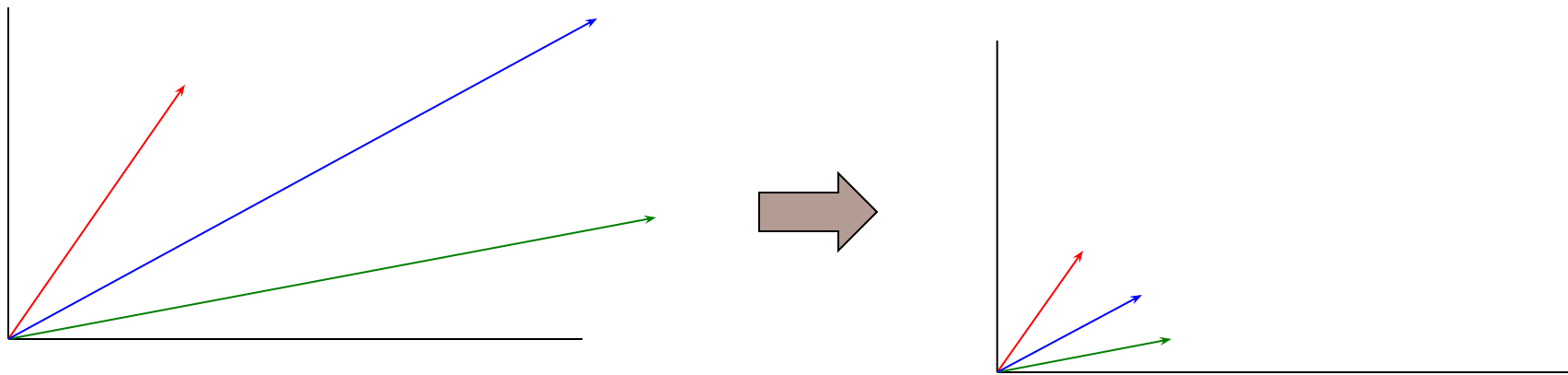


ISSUES WITH K-MEANS: DISTANCE

Cosine similarity

$$\text{sim}(x, y) = \frac{x \bullet y}{|x||y|} = \frac{x}{|x|} \bullet \frac{y}{|y|} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}}$$

correlated with the angle between two vectors



ISSUES WITH K-MEANS: DISTANCE

Cosine Similarity

- Ranges from 0 and 1
- Good for text data and many other real world data
- Computationally friendly since we only need to consider features that have non-zero values for both examples

Cosine Distance

$$d(x, y) = 1 - \text{sim}(x, y)$$

QUESTIONS?

